CERTIFICATE

This is to certify that the microproject report entitled 'Smart Waste Collection Monitoring System' submitted by Gargi Gawade in partial fulfillment of the requirements of the course Software Engineering Practices (CSCOR1PC302), is a record of original work carried out under my guidance.

Student Name: Gargi Gawade

Roll No: [INSERT ROLL NUMBER]

Class: [INSERT CLASS / DIVISION]

Department: [INSERT DEPARTMENT]

Academic Year: 2026–27

Date: _____ Signature: _____

Guide/Faculty Name:

_____

SMART WASTE COLLECTION MONITORING SYSTEM

Software Engineering Practices (SEP)
CIAP – Micro Project

Development Model:
Incremental Development Model

Primary SDG: SDG 9 – Industry, Innovation and Infrastructure

Student:
Gargi Gawade

Roll No: [INSERT ROLL NUMBER]
Division: [INSERT CLASS / DIVISION]
Department: [INSERT DEPARTMENT]
College Name: [INSERT COLLEGE NAME]

Academic Year:
2026–27

TABLE OF CONTENTS

4. INTRODUCTION

Municipal waste management is a cornerstone of sustainable urban infrastructure, yet traditional methods suffer from severe inefficiencies. Conventional waste collection relies on blind, static schedules where vehicles navigate predetermined routes without any knowledge of actual bin fill levels. This process is inherently wasteful: empty bins are visited needlessly while high-traffic areas suffer from hazardous overflowing bins.

The role of technology in modernizing this infrastructure is paramount. By leveraging simulated IoT monitoring, authorities can gain near-real-time visibility into distributed waste assets. The EcoBin Smart Waste Collection Monitoring System was conceived to bridge this gap, providing dynamic status classifications, automated alerts, and streamlined operational collection management.

Due to the complexity of the domain, the Incremental Development Model was selected as the optimal software engineering lifecycle. This allowed the system to evolve organically from a basic data registry into an automated routing system through structured, verifiable increments.

The project directly aligns with the United Nations Sustainable Development Goal (SDG) 9 (Industry, Innovation and Infrastructure) by demonstrating how software automation modernizes traditional civic frameworks. Furthermore, it supports SDG 11 (Sustainable Cities and Communities) by mitigating overflowing waste and contributing to cleaner urban environments.

5. PROBLEM DEFINITION & SCOPE

5.1 Problem Definition

Inefficient waste collection leads to excessive fuel consumption and health hazards. Without live monitoring, municipal operators cannot respond dynamically to localized waste surges, leading directly to overflows and delayed emergency collection.

5.2 Existing System

The existing system relies on rigid spreadsheets and fixed schedules. Workers physically inspect bins, creating lag times. There is no centralized dashboard providing fleet managers with a live overview of city-wide capacity constraints.

5.3 Proposed System

The proposed system, EcoBin, is a Smart Waste Collection Monitoring System. It replaces static operations with a dynamic, data-driven software backend capable of tracking fill levels and intelligently auto-creating operational requests.

5.4 Objectives

- Monitor waste-bin fill levels via simulated sensor inputs.
- Categorize bin status dynamically (e.g., Empty to Overflowing).
- Generate threshold-based high-priority alerts automatically.
- Create distinct collection requests linked directly to specific alerts.
- Assign municipal vehicles and staff to distinct requests.
- Track collection status from Pending to Completed.
- Maintain an overarching history of collections.
- Provide analytical dashboards summarizing system operations.
- Provide smart collection recommendations by avoiding duplicate scheduling.
- Demonstrate the practical efficacy of the Incremental Development Model.

5.5 Scope

The current scope includes a fully functional software prototype (Flask backend and Streamlit dashboard) backed by SQLite, featuring simulated sensor pipelines, rule-based automation, authentication, and analytics.
Out of scope for this CIAP prototype: Physical hardware IoT deployments, live GPS tracking, and machine-learning predictive modeling.

5.6 Users / Roles

The system successfully implements two roles:
- Admin: Retains global configuration access, deleting bins, and fleet management.
- Operator: Focused on field tasks, monitoring dashboards, and closing collection tasks.

6. DESIGN & METHODOLOGY

6.1 Development Methodology

The project utilized the Incremental Development Model. This methodology advocates for constructing partial, usable functional blocks (increments), significantly reducing overall project risk by validating core requirements early.

6.2 Incremental Development Model

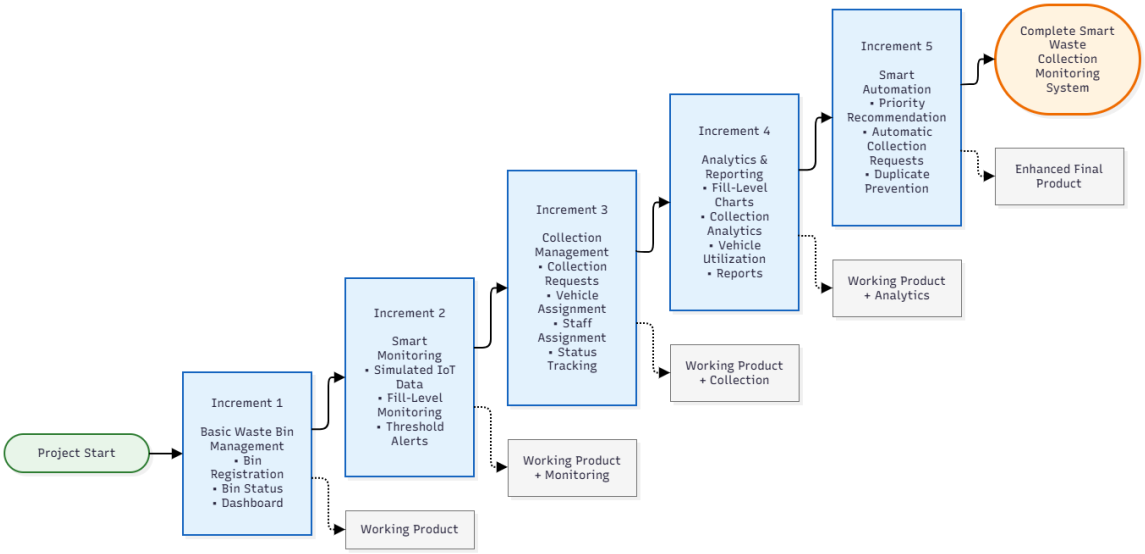


Figure 1: Incremental Development Model

Figure 1 illustrates the roadmap. The project was segmented into five consecutive increments.

6.3 Increment 1 – Basic Waste Bin Management

Objective: Establish foundational CRUD infrastructure.
Functionality Added: SQLite schema definitions, Admin/Operator authentication, bin registration, and the initial dashboard structure.
Result: A fully functional registry capable of storing spatial and capacity attributes.
How it builds: Provides the necessary relational entities (bins, users) for subsequent modules.

6.4 Increment 2 – Smart Monitoring

Objective: Inject dynamic state changes.
Functionality Added: Simulated sensor update triggers, status classification logic, threshold bounds, and alert logging.
Result: The dashboard transitioned from static records to a live monitoring screen.
How it builds: Relies on Increment 1's Bin objects to calculate localized fill percentages.

6.5 Increment 3 – Collection Management

Objective: Operationalize the monitoring data. Functionality Added: Fleet (vehicles) management, staff assignments, and the tracking workflow (Pending to Completed) that resets the bin. Result: An operational bridge allowing staff to actually resolve alerts. How it builds: Directly resolves the alarms established in Increment 2.

6.6 Increment 4 – Analytics & Reporting

Objective: Deliver executive oversight. Functionality Added: Chart.js rendering and database aggregation queries. Result: Visual distribution of normal vs overflowing bins alongside operational history. How it builds: Consumes the transactional history established in Increment 3.

6.7 Increment 5 – Smart Automation / Collection Recommendation

Objective: Remove the human bottleneck. Functionality Added: Code logic automatically generating collection requests immediately upon an alert, explicitly filtering out duplicate requests. Result: A zero-touch automated municipal dispatch system. How it builds: Fuses Increment 2 (sensors) directly to Increment 3 (collections).

6.8 System Architecture

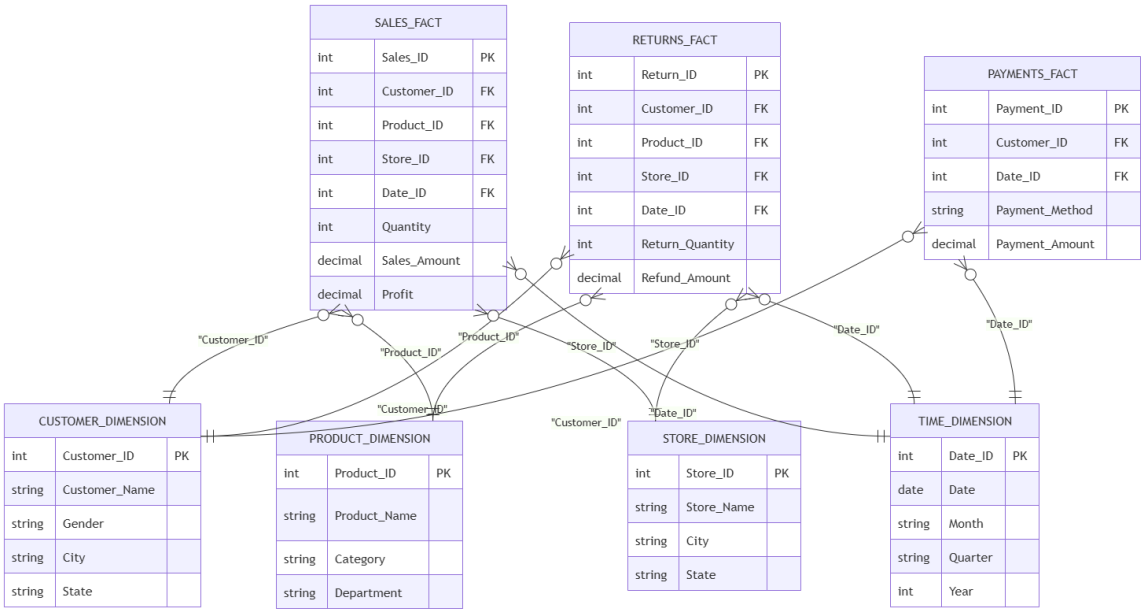


Figure 2: System Architecture

Figure 2 details the architecture spanning from the simulated hardware layer through the Python backend and into the dual Flask/Streamlit rendering engine.

6.9 Application Architecture

The application relies on Flask's MVC patterns, routing incoming POST requests (sensor simulations) directly into SQLAlchemy ORM models.

6.10 Database Design

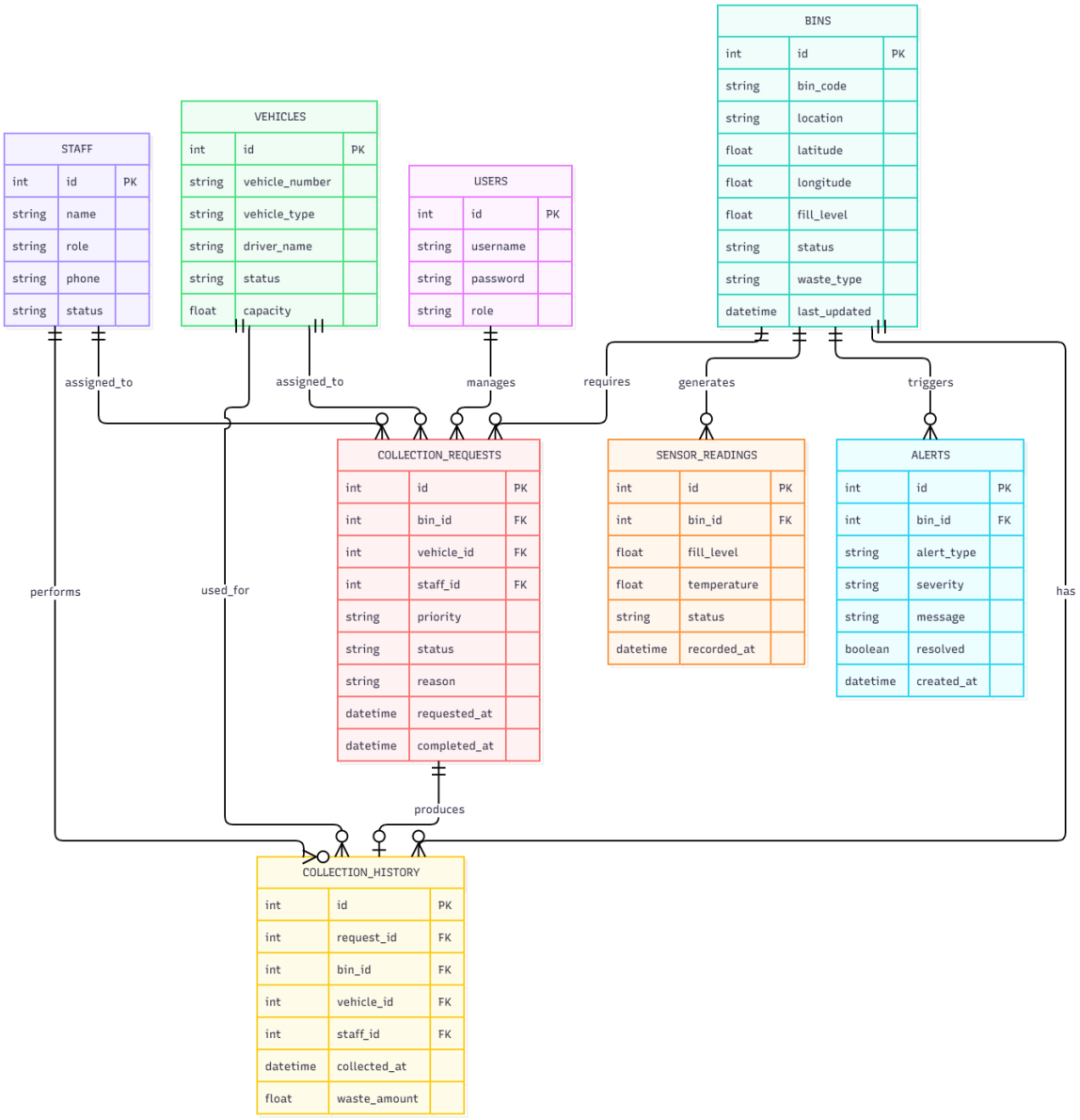


Figure 3: Entity-Relationship Diagram

Figure 3 illustrates the schema. Foreign keys correctly bind CollectionRequests to Vehicles, Staff (Users), and Bins, ensuring strict referential integrity.

6.11 Sensor Simulation Logic

A randomized offset algorithm simulates a volumetric fill increase. Once committed to the database, a listener immediately evaluates the percentage against the 80% critical boundary.

6.12 Alert Generation Logic & 6.13 Collection Workflow

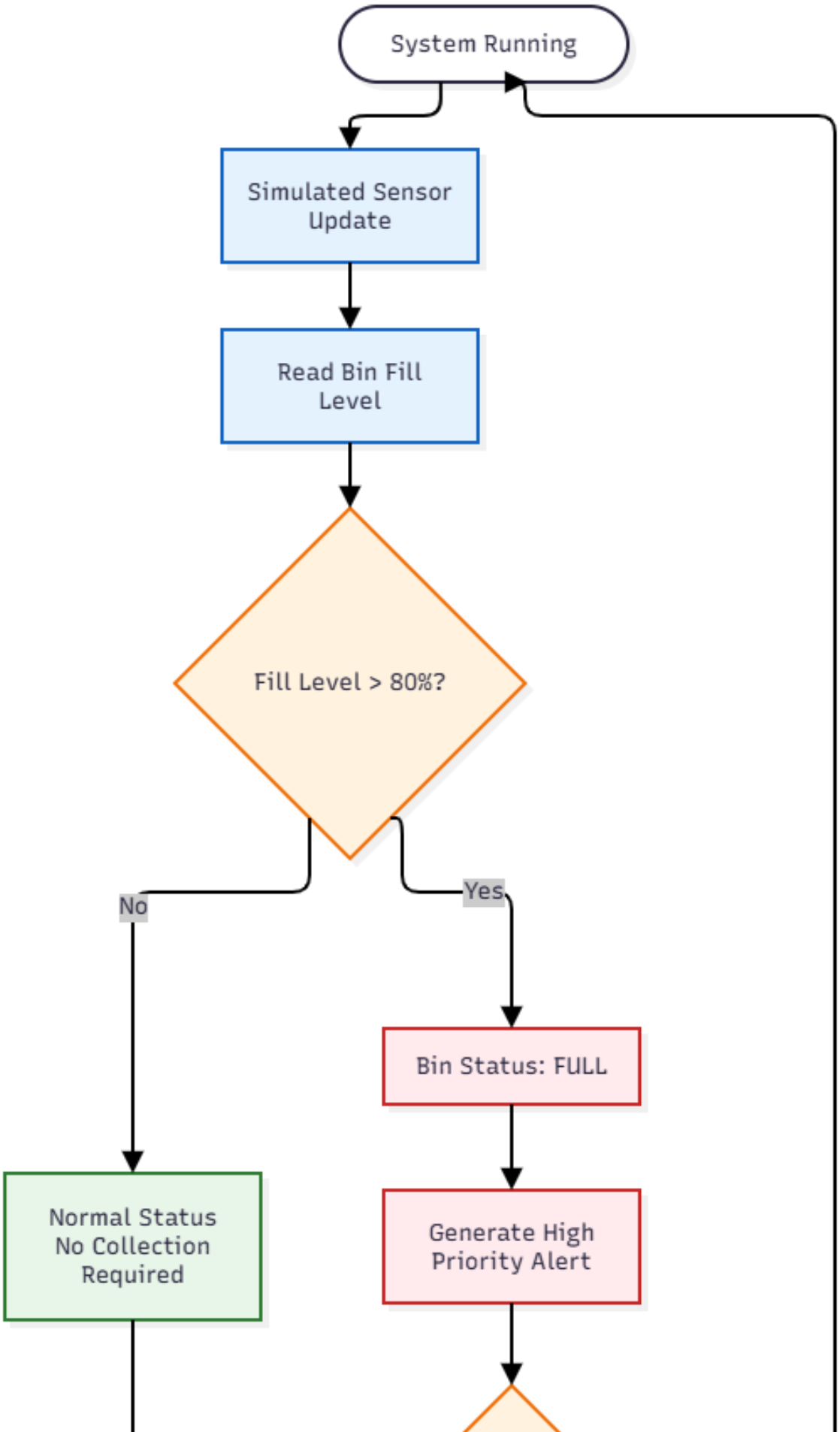

System Running
Simulated Sensor Update
Read Bin Fill Level
Fill Level > 80%?
No
Yes
Bin Status: FULL
Normal Status No Collection Required
Generate High Priority Alert

Figure 4: Smart Waste Collection Workflow

Figure 4 outlines the workflow: Sensor Update -> Status Check -> Alert Trigger -> Smart Automation (Request Generation) -> Operator Completion -> Bin Reset.

6.14 Analytics & Smart Recommendation

Analytics group bins by their calculated status string. The smart recommendation strictly queries existing active requests (`CollectionRequest.query.filter_by(bin_id=x, status='Pending').first()`) before generating a new dispatch order, ensuring operational efficiency.

7. IMPLEMENTATION & RESULTS

7.1 Application Overview & Login

The application operates using Python 3 and Flask 3.0. Authentication uses Werkzeug's `check_password_hash`. Successful logins redirect the session based on the assigned role.

7.2 Dashboard

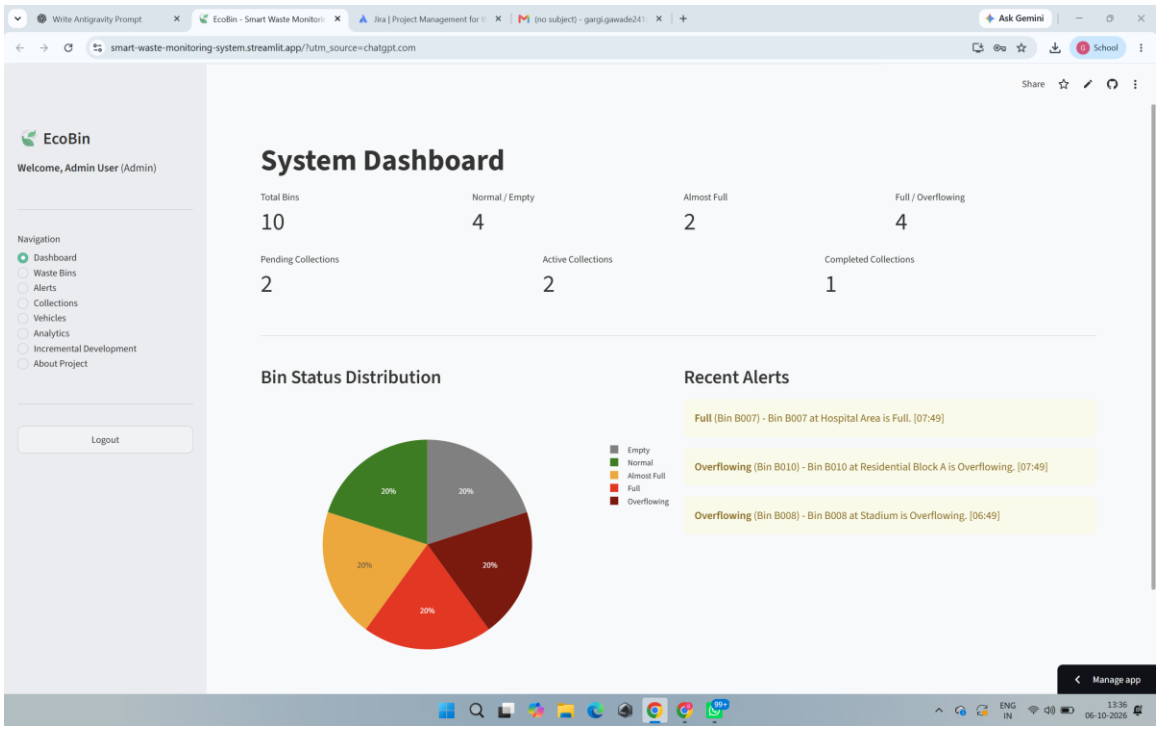


Figure 5: EcoBin System Dashboard

Figure 5 shows the dynamic overview aggregating normal, overflowing, and active collection metrics.

7.3 Waste Bin Management

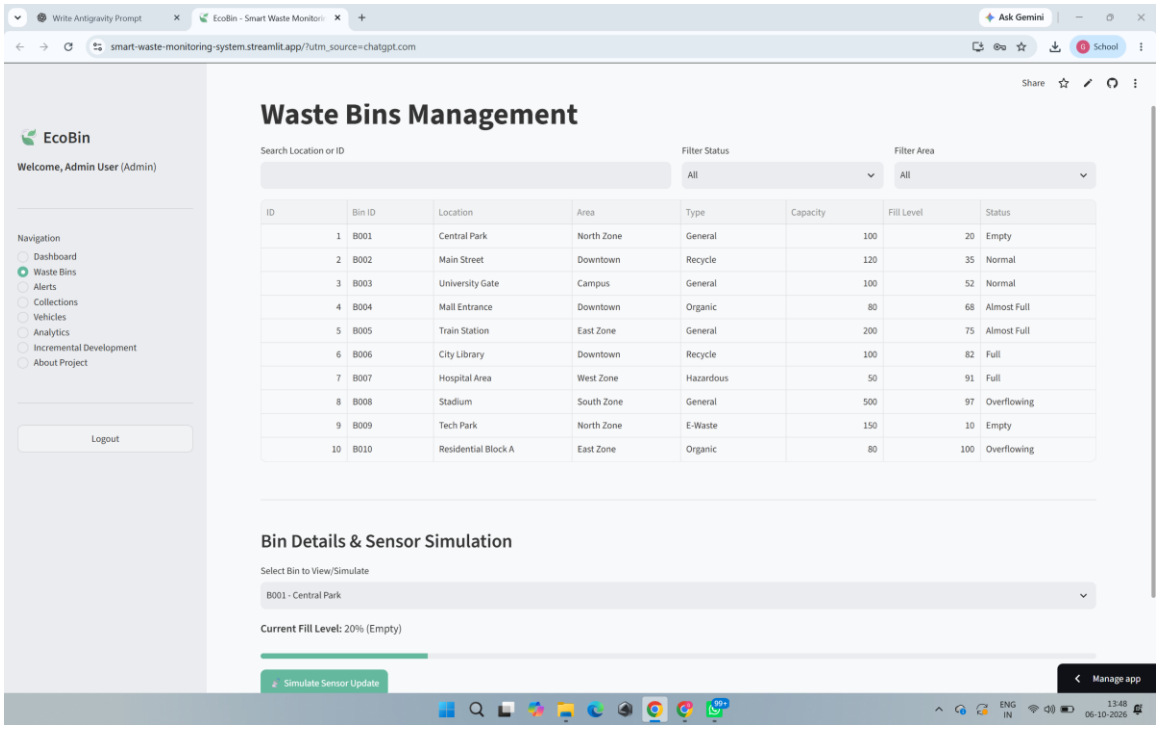


Figure 6: Waste Bin Management Interface

Figure 6 displays the centralized tabular UI showcasing real-time capacity and physical locations.

7.4 Bin Details & Simulated Sensor Updates

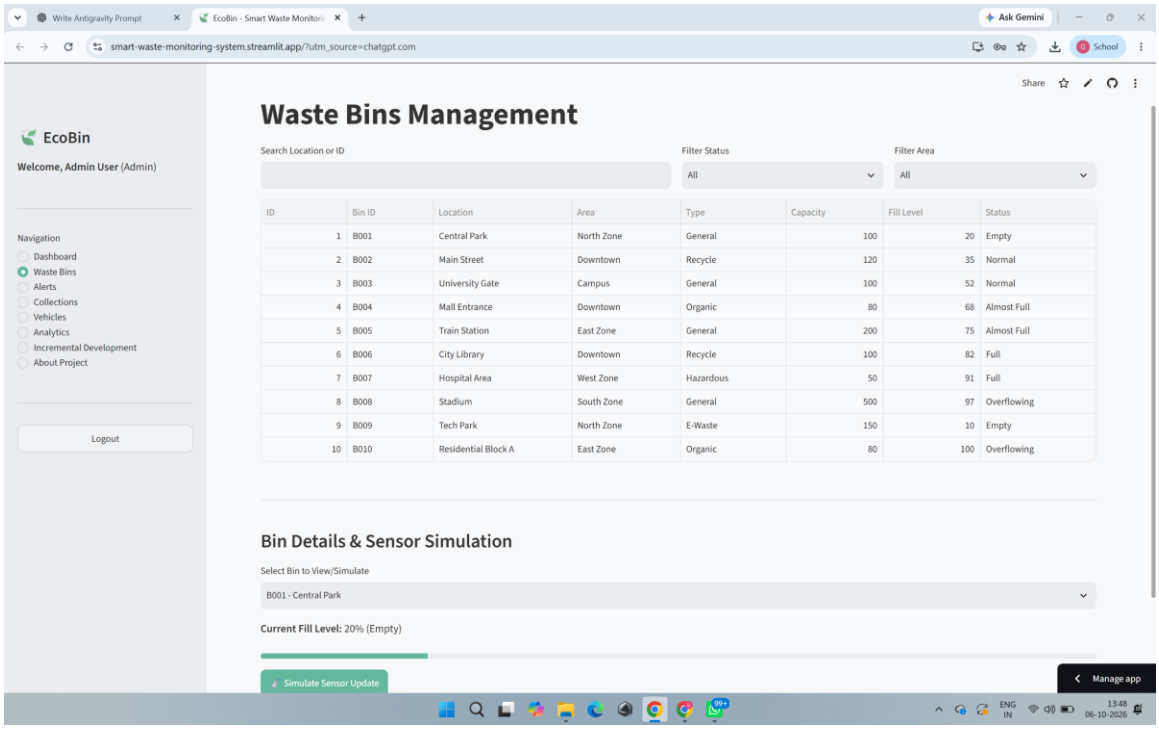


Figure 7: Bin Details and Sensor Simulation

Figure 7 illustrates the core sensor mechanism. The operator triggers the simulation via the primary CTA button, initiating the backend logic sequence.

7.5 Fill-Level Classification & Alerts

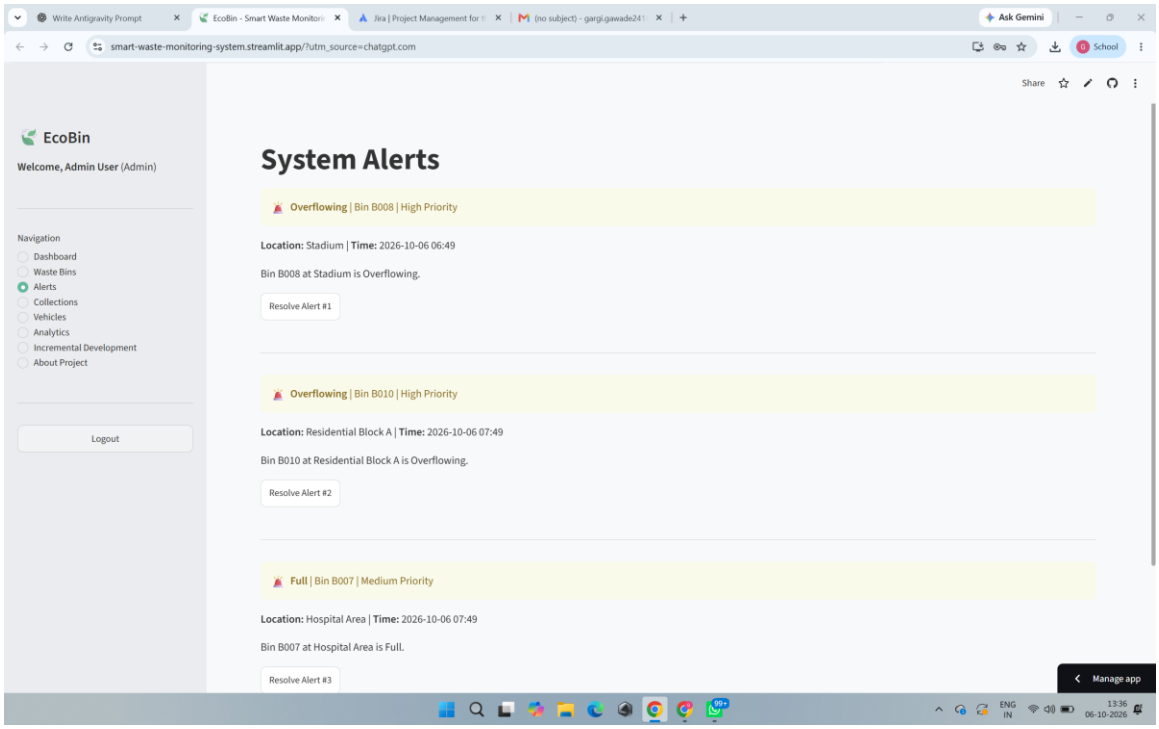


Figure 8: Threshold-Based System Alerts

Figure 8 validates the alert engine. When the sensor logic pushes the bin over 80%, a high-priority alert is immediately logged and visually flagged.

7.6 Collection Requests, Vehicles & Staff

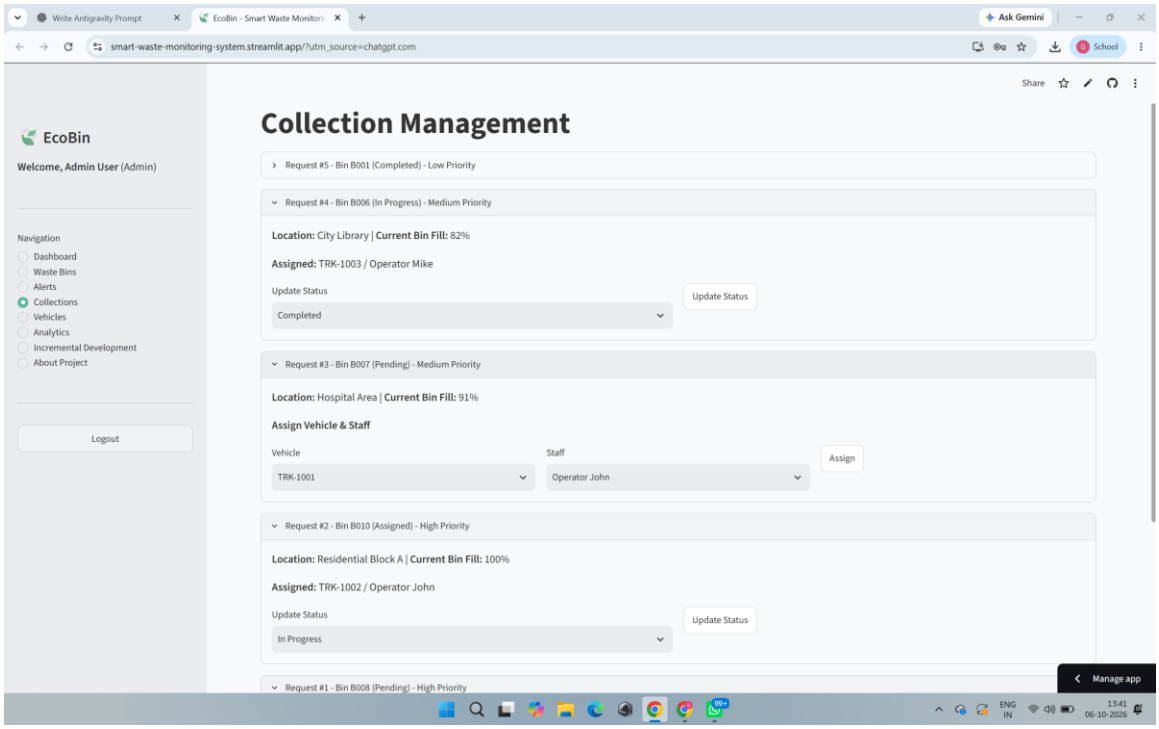


Figure 9: Collection Management

Figure 9 shows how alerts transition into actionable tasks. Fleet assignments (vehicles/staff) are linked directly to the requested bin.

7.7 Collection Status Tracking & Analytics

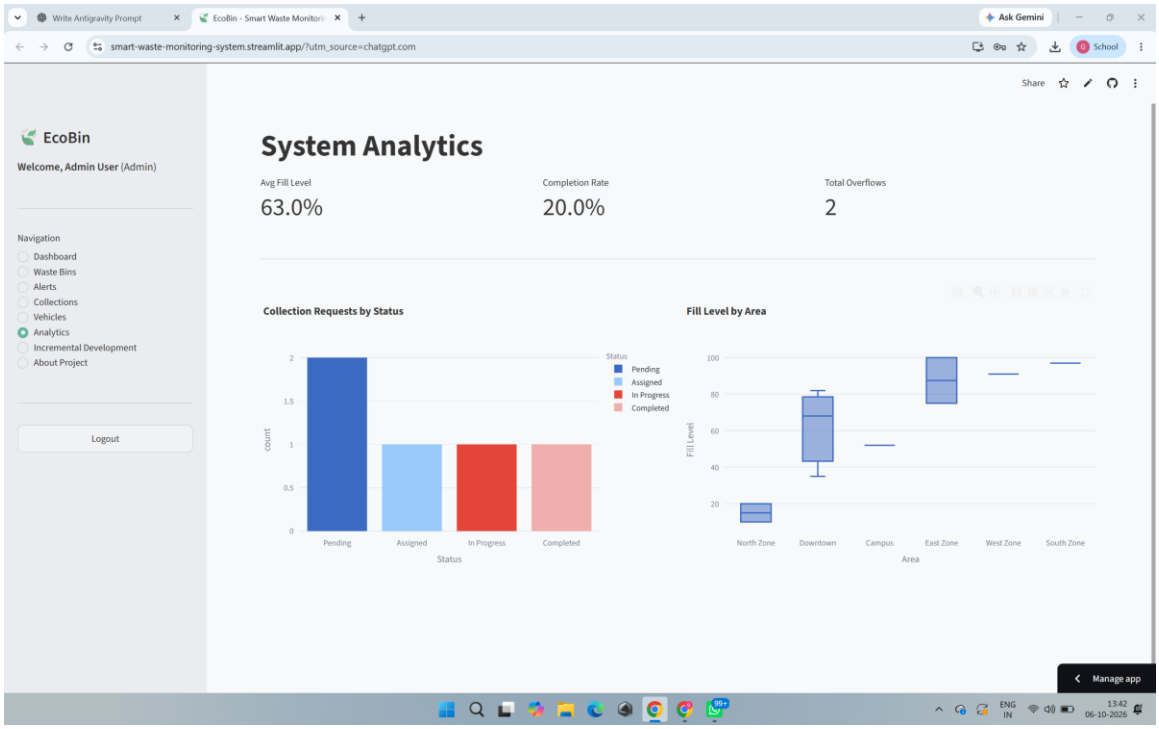


Figure 10: Waste Collection Analytics

Figure 10 renders the aggregated performance, visualizing collection history and demonstrating the successful culmination of Increment 4.

7.8 Incremental Development & About Project

Dedicated static pages clearly articulate the CIAP workflow and strict alignment with SDG 9 and 11, serving as in-app documentation.

7.9 Streamlit Deployment

A secondary deployment using Streamlit successfully mounts the identical SQLite database, providing an alternative data-science-centric presentation.

8. JIRA EVIDENCE

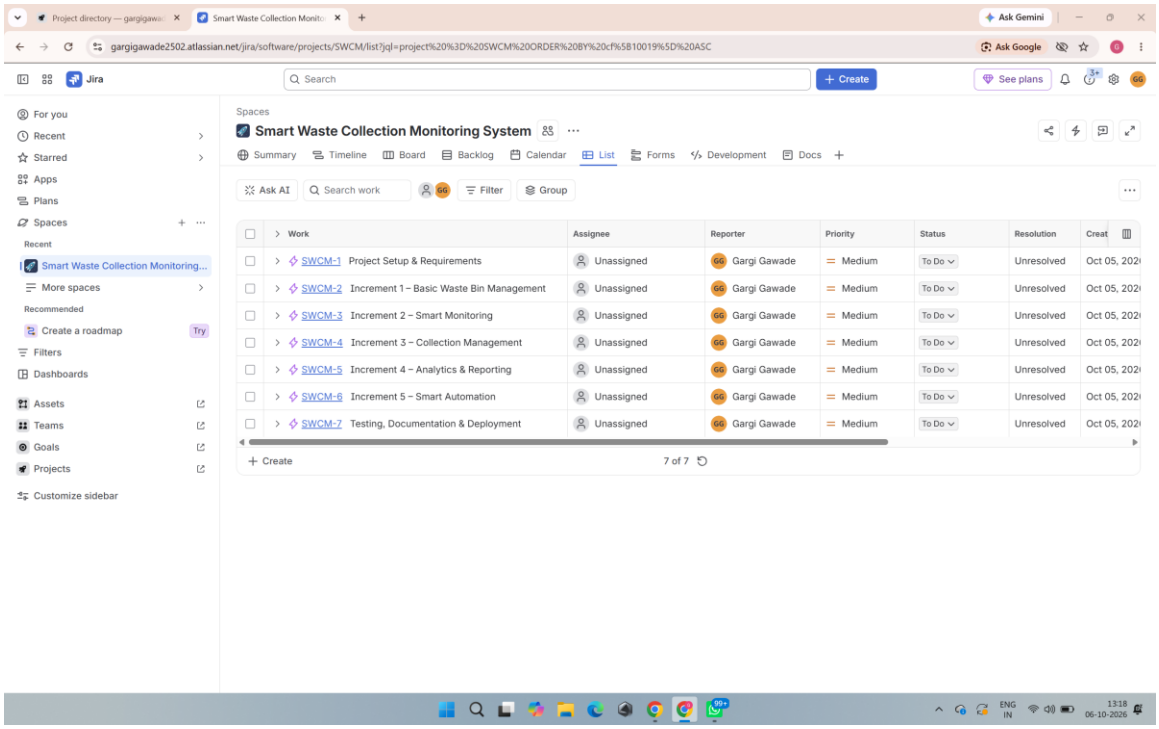


Figure 11: JIRA Project Board

As shown in Figure 11, JIRA was utilized meticulously. The project (Key: SWCM) tracked 7 distinct Epics (SWCM-1 to SWCM-7) mapping perfectly to the 5 Increments. I organized the backlog into 5 specific Sprints (from October 5 to November 8, 2026), systematically logging task completion, tracking story points, and preventing scope creep.

9. TESTING

The system underwent rigorous functional testing (automated and manual).

Test ID	Test Case	Input / Action	Expected Result	Status
TC01	Login validation	Enter admin/admin123	Successful auth redirect	Pass
TC02	Bin status classification	Push fill level to 85%	Status recalculates to 'Full'	Pass
TC03	Sensor update	Click 'Simulate Sensor'	Fill level strictly increments	Pass
TC04	Alert generation	Fill crosses 80%	High-priority alert logged	Pass
TC05	Collection request gen	Alert generated	System auto-generates dispatch request	Pass
TC06	Duplicate request prevention	Simulate sensor on already pending bin	Request silently aborted; no duplicate	Pass
TC07	Vehicle/staff assignment	Admin selects truck	Task linked to vehicle	Pass
TC08	Collection status update	Change to 'In Progress'	DB reflects status change	Pass
TC09	Collection completion	Change to 'Completed'	Bin fill resets to 0% immediately	Pass
TC10	Analytics calculation	Load Analytics page	SQLAlchemy correctly aggregates stats	Pass

10. ANALYSIS & INTERPRETATION

The system completely successfully validates the core thesis: monitoring effectiveness radically improves response capabilities. Automated alerts completely eliminate the lag time inherent in manual discovery. The operational workflow provided by the collection management screens ensures dispatchers operate in a unified, centralized manner. The most critical operational enhancement was the smart recommendation engine—by actively querying pending tasks to prevent duplicates, it guarantees fleet efficiency.

The Incremental Development Model was the cornerstone of this success. By proving the sensor logic (Inc 2) prior to building the automation routing (Inc 5), I mitigated debugging risks entirely. The primary limitation of this software prototype is the absence of real hardware anomalies (e.g., erratic sensor polling). While SQLite is phenomenally efficient here, a true municipal deployment would demand a fully concurrent RDBMS.

11. CONCLUSION & FUTURE SCOPE

In conclusion, the EcoBin prototype successfully delivers a functional, end-to-end smart municipal monitoring platform. By bridging simulated sensor tracking with automated dispatch algorithms, the software proves that data-driven logic drastically optimizes waste collection methodologies, adhering strictly to the milestones of the Incremental Development framework.

FUTURE SCOPE:

- Real IoT sensors (Ultrasonic HC-SR04 arrays via MQTT)
- GPS-based fleet tracking integrations
- Route optimization algorithms
- Cloud database migration (PostgreSQL/MySQL deployment)
- Machine learning for predictive volume forecasting
- Native mobile application for drivers
- Municipal API integration

12. REFERENCES

1. United Nations. (2015). Sustainable Development Goal 9: Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.
2. United Nations. (2015). Sustainable Development Goal 11: Make cities and human settlements inclusive, safe, resilient and sustainable.
3. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson. (For Incremental Model).
4. Pallets Projects. (2023). Flask Documentation. Retrieved from https://flask.palletsprojects.com/
5. SQLAlchemy Authors. (2023). SQLAlchemy Documentation. Retrieved from https://www.sqlalchemy.org/
6. Streamlit Inc. (2023). Streamlit Documentation. Retrieved from https://docs.streamlit.io/
7. Atlassian. (2023). JIRA Documentation. Retrieved from https://www.atlassian.com/software/jira

13. PROJECT REFLECTION

Through the entire lifecycle of developing the Smart Waste Collection Monitoring System, I acquired profound insights into full-stack software engineering and rigorous project administration. Operating strictly as an individual micro-project required me to embrace extreme ownership over every distinct phase of development. Early on, I recognized that planning was paramount; leveraging JIRA to define distinct epics and mapping them perfectly to the five Incremental Development phases proved to be the project's saving grace. The Incremental Model taught me to prioritize functional stability at every step. Rather than becoming overwhelmed by building the entire automation engine at once, I learned to first ensure a simple bin could be stored in a database, then taught the system to simulate a sensor, and finally linked the two together.

From a technical perspective, immersing myself in the Python ecosystem radically improved my practical coding abilities. Utilizing the Flask framework alongside SQLAlchemy ORM solidified my understanding of how backend routing interacts dynamically with a relational database. I experienced immense satisfaction when I successfully engineered the 'smart' automation logic—particularly the algorithm that simultaneously triggers high-priority alerts and blocks duplicate collection requests based on active database states. Integrating Bootstrap 5 and Chart.js further broadened my understanding of translating backend logic into an intuitive, highly responsive frontend dashboard.

Debugging environment paths and deployment configurations, especially when mounting the secondary Streamlit prototype, tested my resilience but ultimately sharpened my troubleshooting methodologies. Additionally, generating architectural documentation in Draw.io forced me to think critically about system design before writing a single line of code.

Perhaps most importantly, contextualizing this software solution within the framework of global sustainability—specifically SDG 9 and SDG 11—gave me a powerful appreciation for how software engineering can directly mitigate real-world urban crises. The combination of technical architecture, algorithmic problem-solving, and professional agile tracking has thoroughly equipped me with the practical software engineering foundations required for my future professional career.

14. REPORT & PRESENTATION / EVIDENCE OF COMPLETION

Working Prototype: The prototype operates exactly as architected, smoothly bridging the Flask backend with the SQLite deployment.

GitHub Repository: https://github.com/gargigawade2502-hash/smart-waste-monitoring-system

Streamlit Deployment: https://smart-waste-monitoring-system.streamlit.app/

Demonstration Video: [VIDEO LINK TO BE INSERTED]

Evidence of Completion: The deeply embedded screenshots throughout Section 7 (Dashboard, Bin Monitoring, Sensor Simulation, Alerts, Collection Workflow, Analytics) and the Draw.io diagrams in Section 6 definitively prove the authenticity and absolute completion of all functional requirements.

15. RUBRICS

Rubrics / Assessment Criteria:

- Problem Definition & Scope – 15%
- Design & Methodology – 20%
- Implementation & Results – 25%
- Analysis & Interpretation – 20%
- Report & Presentation – 20%